2026 EDITION

Network Security Laboratory

Lab 07 · VPN with WireGuard

Tunnel · Split vs Full · Trust as a Routing Decision

University of Calabria · DeMaCS · A.Y. 2025/2026

Before We Start

GNS3 lab — two Ubuntu nodes and one Rasbpian on a managed switch. Boot the topology before the lesson.

Install per role (one-shot apt)

```
# ALICE   (VPN client)
sudo apt install wireguard wireguard-tools dnsutils

# BOB     (VPN server)
sudo apt install wireguard wireguard-tools dnsmasq iptables

# DARTH   (attacker)
sudo apt install tshark dsniff
```

Sanity check before ES 01

```
# on Alice / Bob
wg --version
ip a show eth0     # confirm LAN IP

# on Darth
ip link show eth0 # note your MAC

# enable kernel forwarding only on Darth in ES 0:
# sudo sysctl -w net.ipv4.ip_forward=1
```

Alice — 10.0.0.10

VPN client · 10.10.10.2 inside tunnel

Bob — 10.0.0.20

VPN server · 10.10.10.1 inside tunnel

Darth — 10.0.0.30

On-LAN attacker (MITM in ES 0)

Topology: R1 – SW1 (managed) – {Alice, Bob, Darth}

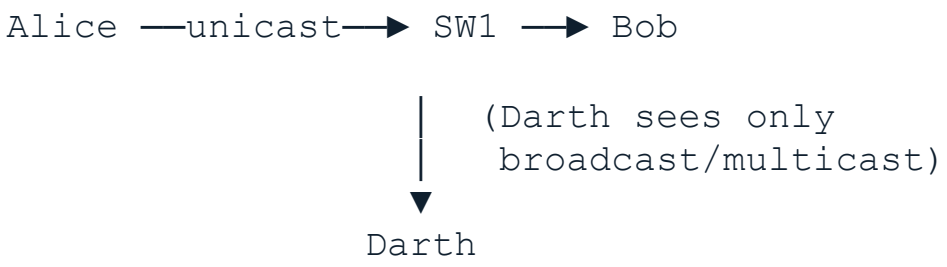
Today's Roadmap

E0	**Darth Gets in Position — MITM via ARP Spoofing** Managed switch ≠ hub. Reuse Lab 04 ARP spoofing on TWO pairs (peer + gateway) plus ip_forward to become an invisible router.
L1	**ES 01 — Payload Protection** Build the WireGuard tunnel. Alice - Bob now talk over an encrypted UDP/51820 stream.
L2	**ES 02 — Measure What L1 Does NOT Protect** Capture before/after the tunnel. Payload disappears — but volume, timing, peer identities stay in clear. Traffic analysis.
L3	**ES 03 — DNS Through the Tunnel** dnsmasq on Bob, dig @10.10.10.1 from Alice. Same query, two routing paths, two visibility profiles.
L4	**ES 04 — Split Tunnel vs Full Tunnel** AllowedIPs = 0.0.0.0/0 + NAT on Bob. Darth sees only encrypted UDP — but Bob now sees everything Alice does.

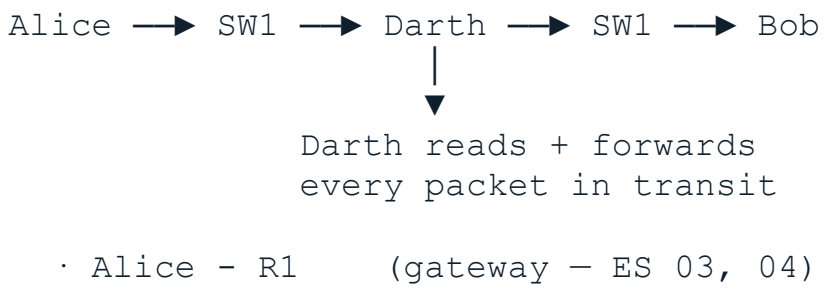
The Managed-Switch Problem

Passive sniffing on a switch does not work. Before defending we have to honestly put the attacker in position.

Before (no MITM)



After (ARP-spoofed MITM)



· Alice - R1 (gateway — ES 03, 04)

From ES 01 onward, Darth is an active man-in-the-middle. The honest question: even from there, what does WireGuard hide?

WireGuard — A Trust Model in One Page

No CA, no certificates. Each peer is one [Interface] + one [Peer] block — literally readable in a screen of text.

1 · KEYPAIR

```
wg genkey | tee privatekey
| wg pubkey > publickey
```

Two ChaCha20-Poly1305 / Curve25519 keys. The private one stays. The public one is the peer's identity.

2 · AUTHORIZE

```
[Peer] PublicKey = …
```

Authorization is membership in a config file. No PKI, no CSR, no expiration. Wrong key → silent drop.

3 · ALLOWEDIPS

```
AllowedIPs = 10.10.10.1/32
```

Dual role: 1) route table for outbound; 2) accept-filter for inbound. The single most important line.

4 · TRANSPORT

```
Endpoint = 10.0.0.20:51820
(UDP)
```

No TCP-over-TCP meltdown. No retransmission at tunnel layer — let the inner protocol handle reliability.

Exercise 1 — Layer 1: Payload Protection

Try on your own. Goal: replace plaintext on the wire with opaque UDP/51820 between Alice and Bob.

1a Generate keypairs (both peers)

```
# on ALICE and on BOB
wg genkey | tee privatekey | wg pubkey > publickey
cat privatekey       # keep secret
cat publickey        # share with peer
# exchange pubkeys between the two consoles
```

1b /etc/wireguard/wg0.conf · BOB

```
[Interface]
PrivateKey = <bob-priv>
Address    = 10.10.10.1/24
ListenPort = 51820

[Peer]
PublicKey  = <alice-pub>
AllowedIPs = 10.10.10.2/32
```

1c /etc/wireguard/wg0.conf · ALICE

```
[Interface]
PrivateKey = <alice-priv>
Address    = 10.10.10.2/24

[Peer]
PublicKey  = <bob-pub>
Endpoint   = 10.0.0.20:51820
AllowedIPs = 10.10.10.1/32
PersistentKeepalive = 25
```

1d Bring up and verify (Bob first)

```
sudo chmod 600 /etc/wireguard/wg0.conf
sudo wg-quick up wg0
sudo wg show            # latest handshake must be recent

# on ALICE
ping 10.10.10.1 -c 4

# on DARTH — only opaque UDP/51820
sudo tshark -i eth0 -Y "udp.port == 51820"
```

Variation — put a wrong PublicKey in [Peer] and try wg-quick up. Does WireGuard error or just silently drop?

L1 Wrap-Up — What L1 Does NOT Protect

Encryption is not anonymity. The right column is what metadata-based traffic analysis lives on.

Darth NO longer sees	Darth STILL sees
• Content of Alice - Bob messages • Inner application protocol (ICMP, HTTP, SSH, …) • Any data destined to 10.10.10.0/24	• Source / destination IPs (10.0.0.10 ↔ 10.0.0.20) • Volume of traffic and timing pattern • The fact that WireGuard is in use (UDP/51820) • Anything NOT routed through the tunnel (DNS, LAN, Internet)

Long download ≠ chat ≠ video call — traffic patterns are recognisable even when encrypted. ES 02 measures this.

Exercise 2 — Layer 2: Measure What L1 Misses

Same destination, two captures (no-VPN, with-VPN). Compare what Darth sees. No new protection — only measurement.

2a Capture WITHOUT VPN

```
# on ALICE
sudo wg-quick down wg0

# on DARTH
sudo tshark -i eth0 -w /tmp/no_vpn.pcap

# on ALICE  (Ctrl+C on Darth when done)
ping 10.0.0.20 -c 5
```

2b Capture WITH VPN

```
# on BOB then ALICE
sudo wg-quick up wg0

# on DARTH
sudo tshark -i eth0 -w /tmp/with_vpn.pcap

# on ALICE — traffic now goes through tunnel
ping 10.10.10.1 -c 5
```

2c Compare the two pcaps

```
# on DARTH
echo "=== WITHOUT VPN ==="
sudo tshark -r /tmp/no_vpn.pcap -T fields -e ip.src -e ip.dst
-e _ws.col.Protocol

echo "=== WITH VPN ==="
sudo tshark -r /tmp/with_vpn.pcap -T fields -e ip.src -e
ip.dst -e _ws.col.Protocol
```

2d Look at raw bytes (variation)

```
# on DARTH
sudo tshark -r /tmp/with_vpn.pcap -x | head -50

# Any readable substring?  Any repeating pattern?
# Answer: no — but the source/dest IPs are still
# perfectly visible in every frame header.
```

Question for the class — after 5 minutes of captured traffic, what can Darth still infer about Alice?

Exercise 3 — Layer 3: DNS Through the Tunnel

Run a resolver inside the tunnel (Bob) and query it explicitly from Alice. Three ubuntu pitfalls to handle.

3a Reproduce the leak (default resolver)

```
# on DARTH
sudo tshark -i eth0 -Y "dns" \
  -T fields -e ip.src -e ip.dst -e dns.qry.name

# on ALICE  (tunnel already up, NO DNS= in wg0.conf)
dig google.com
dig github.com
# domains appear in clear on Darth's wire
```

3b Free port 53 on BOB

```
# on BOB
sudo apt install dnsmasq

# systemd-resolved holds port 53 — drop it
sudo systemctl stop    systemd-resolved
sudo systemctl disable systemd-resolved

# verify Bob can reach a real upstream
dig google.com @8.8.8.8
```

3c dnsmasq drop-in on BOB

```
# /etc/dnsmasq.d/lab07.conf
no-resolv
server=8.8.8.8
listen-address=127.0.0.1,10.10.10.1
bind-interfaces

# wg0 up FIRST  (10.10.10.1 must exist)
sudo wg-quick up wg0 && systemctl start dnsmasq
```

3d Query the resolver inside the tunnel

```
# on ALICE
dig google.com  @10.10.10.1
dig github.com  @10.10.10.1
# SERVER line in reply must read: 10.10.10.1#53

# on DARTH  — domains gone, only opaque UDP/51820
# on BOB    — sudo tshark -i wg0 -Y "dns"
#             queries readable here  (trust shift)
```

Variation — dig google.com @1.1.1.1 · Does this bypass the fix? What iptables rule on Alice would block it?

Split Tunnel vs Full Tunnel

Two lines of config. Two threat models. Make the choice — don't inherit it.

	SPLIT (ES 01–03)	FULL (ES 04)
What goes in the tunnel	Only 10.10.10.0/24	Everything (0.0.0.0/0)
What stays in the clear	LAN, gateway, Internet, multicast	Nothing (except link-local)
Typical use case	Remote worker → corporate intranet	Privacy VPN (NordVPN, Mullvad), exit-node
Advantage	Local stays local — VPN bandwidth not a bottleneck	Nothing of Alice visible on her LAN
Cost	Partial privacy — non-VPN traffic in clear	Total trust shift to Bob — he sees everything

Wrong tunnel = wrong threat model. A full tunnel does not 'protect more' — it replaces ISP-trust with provider-trust.

Exercise 4 — Layer 4: Full Tunnel

Convert split → full. Two lines change on Alice, NAT on Bob. Then re-run the Step 0 capture and compare.

4a Baseline — split tunnel still leaks LAN

```
# on DARTH — capture non-tunnel traffic from Alice
sudo tshark -i eth0 -Y "ip.src == 10.0.0.10 and not udp.port
== 51820"

# on ALICE
ping 10.10.10.1 -c 3      # invisible to Darth
ping 10.0.0.1   -c 3      # VISIBLE to Darth  ← leak
```

4b Enable forwarding + NAT on BOB

```
# on BOB
sudo sysctl -w net.ipv4.ip_forward=1

# in /etc/wireguard/wg0.conf  ·  [Interface]
PostUp   = "iptables -t nat -A POSTROUTING -o eth0 -j
MASQUERADE"
PostDown = "iptables -t nat -D POSTROUTING -o eth0 -j
MASQUERADE"
```

4c Change AllowedIPs on ALICE

```
# /etc/wireguard/wg0.conf  ·  [Peer] on Alice
AllowedIPs = 0.0.0.0/0

# restart  (Bob first)
sudo wg-quick down wg0 && sudo wg-quick up wg0

# verify default route
ip route show     # default dev wg0 …
```

4d Re-run baseline capture

```
# on DARTH — same filter as 4a
sudo tshark -i eth0 \
  -Y "ip.src == 10.0.0.10 and not udp.port == 51820"

# on ALICE — same two pings
ping 10.10.10.1 -c 3
ping 10.0.0.1   -c 3       # NOW invisible to Darth
# on BOB: outgoing ICMP has src 10.0.0.20 (NAT)
```

Hint — if Internet feels slow, add MTU = 1380 in [Interface] on Alice (WireGuard takes overhead).

WireGuard vs OpenVPN vs IPsec

Same job; very different complexity, trust, and auditability surfaces.

	WireGuard	OpenVPN
Config complexity	1 file, ~10 lines	PKI + certs
Crypto	ChaCha20 + Curve25519	Configurable (TLS)
Kernel integration	Yes (Linux ≥ 5.6)	Userspace
Transport	UDP	TCP or UDP
Trust model	Pre-shared public keys	PKI / certificates
Auditable codebase	~4 000 lines	~100 000 lines

Key Takeaways

L1

Encryption ≠ anonymity

WireGuard hides the payload of Alice ↔ Bob. It does NOT hide who talks to whom, when, or how much.

L2

Metadata is enough

ISPs, ad-networks and governments rely on traffic analysis — pattern, volume, peer identity. The content is often not needed.

L3

Protected = routed through wg0

DNS leaks because the OS resolver was never told to use the tunnel. Fixing it is a routing decision, not a crypto upgrade.

L4

Split vs Full is a threat-model choice

Corporate access wants split. Privacy-against-the-LAN wants full — and trades one observer (LAN) for another (the VPN provider).